CompNeuroVis Architecture Review

Native simulator attachment, multi-mode scientific apps, and comparable framework patterns

Executive summary

- The fundamental architecture is sound if CompNeuroVis is treated as a shared scene, state, runtime, and frontend substrate that supports several authoring modes rather than one universal app DSL.
- The most important correction is that NEURON users should be able to keep programming in NEURON. CompNeuroVis should attach to native simulator objects, observe references, expose controls, add tools, and render views. Higher-level declarative builders should exist as optional conveniences, not as the primary replacement for native simulator programming.
- The broader goal, including NeuroML editing, static visualization, replay, and live simulator control, is coherent. It requires a pluralistic architecture: native simulator attachment, document/editor mode, live app mode, static/replay mode, and headless/export mode should all lower into a common internal model.
- The current pressure point is not the core IR. The pressure point is that examples are forced to manually interact with too many internal concepts: fields, view specs, panel layout, update emission, reset behavior, action routing, selection state, and simulator mutation. The public integration layer should compress this without hiding native simulator code.

Contents

- 1. Core thesis
- 2. What should stay native
- 3. The five app archetypes
- 4. Revised architecture
- 5. Comparable framework findings
- 6. Mapping to CompNeuroVis
- 7. Design implications for the current code
- 8. Practical refactor sequence
- 9. Architecture tests
- 10. References

1. Core thesis

CompNeuroVis should not force one authoring model. It should support multiple public entry points over one shared internal substrate. The internal substrate can remain strongly typed and explicit: Field, Geometry, Scene, ViewSpec, PanelSpec, LayoutSpec, State, Command, Update, Tool, Runtime, Transport, and Frontend. The public layer should differ by use case.

```
Native simulator code     -> Adapter / instrumentation -> Scene + State + Updates -> Frontend
Document/editor code      -> Document session          -> Scene + State + Updates -> Frontend
Static/replay data        -> Scene builder             -> Scene + Fields         -> Frontend
Headless/export workflow  -> Scene serializer/exporter -> Files / images / HTML
```

The architecture becomes unsound only if every use case is forced through one domain-specific session type, especially a NEURON live session. A NeuroML editor, a static morphology viewer, a replay viewer, and a live NEURON tuning surface have overlapping infrastructure, but they do not have the same ontology.

2. What should stay native

For NEURON users, the source of truth should remain native NEURON code. CompNeuroVis should behave more like matplotlib, ipywidgets, Panel, Streamlit, or a Blender add-on: it attaches to an object graph that already exists, rather than forcing the user to translate the model into a new framework-owned language.

```
from neuron import h
from compneurovis import NeuronViewer

# Normal NEURON model construction.
soma = h.Section(name="soma")
soma.insert("hh")
stim = h.IClamp(soma(0.5))
stim.delay = 5
stim.dur = 1
stim.amp = 0.1

# CompNeuroVis attaches on top.
viewer = NeuronViewer(title="Single-cell viewer")
viewer.add_sections([soma])
viewer.trace("soma.v", soma(0.5)._ref_v, unit="mV")
viewer.slider("stim.amp", target=stim, attr="amp", min=0.0, max=1.0)
viewer.run()
```

This style preserves scientific familiarity. A user already fluent in Sections, Segments, PointProcesses, NMODL mechanisms, Vector.record references, h.fadvance, and IClamp should not need to learn a separate model-authoring DSL before they can visualize or interact with the model.

3. The five app archetypes

App archetype	Definition	Example
Native simulator attachment	User writes normal NEURON, Jaxley, MOOSE, OpenMM, or custom Python, then registers morphology, traces, controls, and tools.	A NEURON user adds voltage traces and current-clamp placement to an existing model.
Live interactive simulation app	A simulator-backed app with live ticking, parameter controls, selection, perturbation tools, rolling traces, reset behavior, and custom panels.	The pharynx muscle viewer with EGL-19, EXP-2, CCA-1, gap junctions, and live parameter tuning.
Static or replay visualization	No live simulator required. The app loads arrays, saved traces, morphologies, graph layouts, or simulation output and renders them.	A replay of voltage traces across a morphology, or a static NeuroML morphology browser.
Document/editor application	The root object is an editable document with validation, property inspection, undo/redo, import/export, and optional visual editors.	A NeuroML editor with morphology editing, channel inspectors, connectivity graphs, and validation panels.
Headless/export workflow	Scene and data are generated without an interactive event loop and exported to files, images, JSON, HTML, or reports.	Batch-rendering morphology snapshots or exporting a static web viewer from a saved scene.

The core architectural rule is that these archetypes should be siblings. NEURON live viewing should not be the root class for a NeuroML editor, and a document editor should not pretend to be a simulator session.

4. Revised architecture

The recommended stack has five layers. Lower layers should be small and stable. Higher layers should be ergonomic and domain-specific.

Layer	Responsibility	Examples
Core data/view IR	Domain-neutral representation of values, structure, views, panels, and layout.	Field, Geometry, Scene, OperatorSpec, ViewSpec, PanelSpec, LayoutSpec
Runtime/event substrate	Shared machinery for state, commands, updates, tools, selections, transports, and update loops.	StateStore, Command, Update, ToolManager, SelectionState, Transport, RuntimeLoop
App mode abstractions	Different public app shapes that reuse the substrate without pretending to be the same kind of app.	StaticApp, LiveApp, SimulatorApp, DocumentApp, HybridApp
Domain integrations	Adapters and convenience APIs for domain-specific objects and user expectations.	NeuronViewer, NeuroMLDocument, NeuroMLEditor, JaxleyAdapter, OpenMMViewer
Templates and demos	High-level examples and packaged workflows built on the lower layers.	single_cell_viewer, network_viewer, pharynx_chain_viewer, neuroml_editor

```
compneurovis.core
    Field, Geometry, Scene, ViewSpec, PanelSpec, LayoutSpec

compneurovis.runtime
    StateStore, Command, Update, RuntimeLoop, Transport, ToolManager

compneurovis.frontends
    VisPyFrontend, NotebookFrontend, WebFrontend

compneurovis.integrations.neuron
    NeuronViewer, NeuronLiveApp, NeuronAdapter, IClampTool, SectionGeometryMapper

compneurovis.integrations.neuroml
    NeuroMLDocument, NeuroMLEditor, NeuroMLValidator, NeuroMLToNeuronBridge

compneurovis.apps
    Higher-level templates and demos
```

5. Comparable framework findings

The research findings remain relevant, but they should be interpreted through the corrected goal: preserve native programming models while adding optional higher-level tooling.

Framework	Relevant pattern	CompNeuroVis implication
matplotlib	Does not own the numerical model. Users compute arrays however they want, then plot them.	CompNeuroVis should let users keep simulator-native code and register traces, fields, and views.
ipywidgets	Binds frontend controls to Python-side state through traitlets and widget events.	Controls should be easy bindings over existing Python or simulator objects.
Panel / Param	Builds reactive UIs around ordinary Python objects and parameterized state.	Useful model for declarative controls and inspectors without replacing scientific code.
Streamlit	Wraps ordinary Python scripts with widgets and rerun/session-state semantics.	Good authoring philosophy, but its rerun model is too coarse for the live simulation core.
Unity	Users write C# against GameObjects and Components. Prefabs, ScriptableObjects, packages, and input actions add editor/runtime	CompNeuroVis can expose native simulator state through tools and inspectors rather than replacing the underlying model.

	structure.	
Unreal	Actors, Components, reflected UObjects, Blueprints, plugins, modules, and Enhanced Input coexist over the same runtime object model.	Support multiple authoring levels: native code, visual/editor tooling, reusable modules, and action contexts.
Blender	Persistent data-blocks, DNA/RNA metadata, operators, modal tools, and add-ons separate data from transient tool behavior.	Strong model for NeuroML/document editing and interactive tools over persistent scientific documents.
VTK / ParaView	Data objects, algorithms, pipelines, views, state files, Python traces, programmable filters, and client-server execution are separated.	Supports the idea of Scene as a reproducible intermediate state and update pipeline.
VisPy	Separates scenegraph, visuals, canvas backends, cameras, and lower rendering layers.	Validates the separation between view semantics and frontend rendering implementation.
NEURON / LFPy	Raw NEURON is flexible but verbose. LFPy provides higher-level domain wrappers while still relying on NEURON underneath.	CompNeuroVis should offer thin wrappers that reduce visualization boilerplate without replacing NEURON.
Arbor	Separates recipe as model description from simulation as executable instantiation, with convenience wrappers for common cases.	Useful precedent for distinguishing domain model descriptions from runtime execution, but not a reason to replace native NEURON authoring.
OpenMM	Separates System, Force, Integrator, Context, Platform, plugins, and application-layer conveniences.	Good precedent for adapter, runtime context, plugin backends, and optional high-level application APIs.
Brian2	High-level equations and monitors compile through devices and code generation.	Useful for trace/monitor design and code generation, but represents a more framework-owned modeling style.
NEST	Kernel, devices, recording backends, PyNEST, NESTML, and NEST Server separate model, recording, and transport concerns.	Supports swappable recording/transport backends and remote session ideas.

6. Mapping to CompNeuroVis

Abstraction	Recommendation	Rationale
Field	Keep	Domain-neutral labeled data array. Works for traces, static replay data, validation overlays, graph metrics, and morphology annotations.
Geometry	Keep	Domain-neutral structure and spatial embedding. Should support NEURON morphologies, NeuroML morphologies, meshes, graphs, and abstract layouts.
ViewSpec	Keep	Represents visual intent, not renderer implementation. Should remain frontend-neutral.
PanelSpec / LayoutSpec	Keep but evolve	Useful abstraction for visual composition. Long-term, a recursive layout tree will likely be more general than fixed panel grids.
Session	Generalize	Do not equate Session with NEURON live simulation. It should be a runtime/event concept with specialized subclasses or modes.
NeuronSession	Demote to integration	Important, but should live under the NEURON integration path rather than as the conceptual root of the framework.
FieldAppend	Keep as live-data update	Excellent for rolling traces and streaming values, but not the right primitive for document editing.
DocumentPatch	Add	Needed for NeuroML and other editable documents. Should coexist with FieldAppend and ScenePatch.
ControlSpec	Keep but bind higher	Controls should bind to simulator objects, document properties, or StateStore entries without manual if/else dispatch in every example.
ActionSpec / Tool	Keep and clarify	Actions are semantic commands. Tools should be transient interaction modes, closer to Blender modal operators or VTK interactor styles.

7. Design implications for the current code

The pasted pharynx example demonstrates the correct scientific ambition, but it exposes too many framework internals in one subclass. The same class constructs NEURON sections and mechanisms, creates gap junctions, prepares recorders, defines fields, defines views, mutates layout, emits FieldAppend and FieldReplace updates, handles reset logic, declares controls, applies parameter changes, tracks click modes, handles key presses, manages IClamps, and emits status messages.

- The NEURON construction should remain user-owned and familiar.
- The repetitive CompNeuroVis wiring should move into reusable attachment APIs, trace helpers, control bindings, standard tools, and app-mode-specific helpers.
- The app should not manually restate series names and IDs across recorders, fields, views, reset handling, and append handling.
- IClamp placement should be a reusable NEURON tool, not a bespoke interaction state machine in every app.
- The same public API should support simple scripts, larger apps, and later public-facing layers.

```
# Desired direction for the pharynx example.
viewer = NeuronViewer(title="Pharynx muscle viewer", v_init=-73)
secs, halfgaps = build_pharynx_model_native_neuron()

viewer.add_sections(secs, layout="chain")
viewer.voltage_trace(selectable=True, rolling_window=120, y_range=(-90, 70))
viewer.line_trace("exp2_state", refs=pm4_exp2_refs(), names=("C1", "C2", "C3", "O", "I"))
viewer.line_trace("ica", refs=(pm4_seg._ref_ica,), names=("I_Ca",))
viewer.slider("G_EXP2", on_change=lambda v: set_all_segments(secs, "exp2.gmax", v))
viewer.xy_pad("g_egl19_exp2", on_change=set_egl19_exp2)
viewer.add_tool(IClampPlacementTool(secs))
viewer.run()
```

8. Practical refactor sequence

1. **Extract SceneBuilder:** Support static and replay visualization without requiring NeuronSession. It should create fields, geometry, views, panels, and layout cleanly.
2. **Extract NeuronViewer:** Provide native NEURON attachment: add_sections, trace_ref, trace_many, slider, xy_pad, add_tool, run.
3. **Extract standard trace machinery:** A trace declaration should create recorder, field, view, append behavior, retention policy, and reset behavior.
4. **Extract standard control binding:** A control should bind to target/attr, callback, StateStore path, or document property without per-example apply_control if/else branches.
5. **Extract tools as reusable modes:** IClamp placement, trace selection, brush selection, camera interaction, and property editing should be reusable Tool objects.
6. **Introduce DocumentApp:** NeuroML editing needs document model, selection, validation, property inspector, undo/redo, import/export, and optional live preview.
7. **Generalize Session into runtime modes:** Distinguish StaticApp, LiveApp, SimulatorApp, DocumentApp, and HybridApp. Keep shared command/update/transport mechanics.
8. **Add NeuroML integration:** Start with load, display, validate, inspect, and export. Later add live preview through NEURON generation.
9. **Add headless export:** Make Scene serializable and render/export without a live event loop.

9. Architecture tests

The following tests are useful for deciding whether an abstraction belongs in the core or in a domain/app layer.

Test	Pass condition
Can a NEURON user write normal NEURON?	The user can construct Sections, mechanisms, PointProcesses, clamps, and h.fadvance logic normally, then attach CompNeuroVis with a few viewer calls.
Can a static replay avoid simulator concepts?	A saved trace or morphology can be loaded into fields and views without creating a simulator session.
Can a NeuroML editor avoid live-trace semantics?	Document edits, validation, property inspection, and undo/redo work without pretending to emit FieldAppend.
Can a live simulator avoid document-editor concepts?	A parameter-tuning app does not need document patches, undo stacks, or XML-specific validation unless explicitly added.
Can a tool be reused across apps?	IClamp placement, selection, or property inspection can be registered as a Tool instead of copied into each Session subclass.
Can the frontend be swapped?	The same scene/update semantics can drive VisPy, notebook widgets, web UI, or export paths.
Can the transport be swapped?	Local process, websocket, notebook comms, and headless export should not change model code.

10. Recommended decisions

- Keep the core IR small, typed, and domain-neutral.
- Make native simulator attachment the primary scientific-user experience.
- Make declarative builders/templates optional conveniences, not the required modeling style.
- Treat NEURON live viewing, NeuroML editing, static replay, and headless export as sibling app modes.
- Move FieldAppend, FieldReplace, reset handling, rolling buffers, and view construction behind trace helpers.
- Move IClamp placement and selection into reusable tools.
- Add DocumentApp and DocumentPatch concepts before trying to build a serious NeuroML editor.
- Do not allow NeuronSession to become the root ontology of the entire framework.

11. References and comparable systems reviewed

These sources informed the architecture comparison and remain useful design references. They are included as normal references rather than tool-specific citation tokens.

- Unity Manual - Components
- Unity Manual - ScriptableObject
- Unity Input System - Actions
- Unreal Engine - Actors
- Unreal Engine - Components
- Unreal Engine - Blueprints
- Unreal Engine - Plugins
- Blender Developer Docs - IDs / data-blocks
- Blender Developer Docs - DNA
- Blender Developer Docs - RNA
- Blender Python API - Operators

- NEURON Python documentation
- NEURON NMODL documentation
- LFPy classes documentation
- Arbor C++ documentation
- Arbor single-cell model documentation
- OpenMM Developer Guide - Core Library
- OpenMM Developer Guide - Writing Plugins
- Brian2 - How Brian works
- Brian2 - Recording
- NEST documentation
- NEST Server
- ipywidgets low-level widgets
- Panel reactive API
- Streamlit architecture
- Dash layout and callbacks
- VisPy getting started
- VTK Book - Data representation
- ParaView Python and batch documentation

Source note: This document also uses the pasted PharynxMuscleSession example as a concrete case study for boilerplate and role leakage.